

01. 파이썬 기반의 머신러닝과 생태계 이해

▼ 목차

1. 머신러닝의 개념
2. 파이썬 머신러닝 생태계를 구성하는 주요 패키지
3. 넘파이
4. 데이터 핸들링 - 판다스
5. 정리



1. 머신러닝의 개념

머신러닝이란?

- 일반적으로 애플리케이션을 수정하지 않고도 데이터를 기반으로 패턴을 학습하고 결과를 예측하는 알고리즘 기법을 통칭함

< 예시 >

금융 사기 거래를 적발하는 프로그램을 만들자!

→ 복잡한 금융 거래의 수많은 변수에 대해 아주 많은 코드로 된 프로그램을 작성하더라도 금융 사기 전문가들은 교묘하게 보안을 뚫을 것

→ 또한 주변 환경이 시대에 따라 계속해서 변화하기 때문에 변화한 시대상에 맞춰 기존 로직을 다시 수정하고 검증하는 과정은 많은 시간과 비용이 소모됨

→ 이와 같이 업무적으로 복잡한 조건/ 규칙들이 다양한 형태로 결합하고 변화하는 속에서 이들을 관통하는 일정한 패턴을 찾기 어려움

→ 이러한 상황에서 머신러닝이 유용하게 이용 됨

즉, 머신러닝은

1. 해당 문제를 데이터를 기반으로 하여 숨겨진 패턴을 인지해 해결함
2. 머신러닝 알고리즘은 데이터를 기반으로 통계적인 신뢰도를 강화하고 예측 오류를 최소화하기 위한 다양한 수학적 기법을 적용해 데이터 내의 패턴을 스스로 인지하고 신뢰도 있는 예측 결과를 도출해 냄
3. 데이터를 관통하는 패턴을 학습하고, 이에 기반한 예측을 수행 → 데이터 분석 영역에 큰 기여

데이터 분석 영역은 이러한 머신러닝의 영향으로 → 머신러닝 기반의 예측분석으로 발전 중

머신러닝의 분류

지도학습	비지도학습
• 분류 • 회귀 • 추천 시스템 • 시각/ 음성 감지/ 인지 • 텍스트 분석, NLP	• 클러스터링 • 차원 축소 • 강화학습

데이터 전쟁

데이터 vs 머신러닝 알고리즘, 무엇이 더 중요?

머신러닝의 가장 큰 단점은 데이터에 매우 의존적이라는 것

→ 즉, 좋은 데이터를 갖추지 못한다면 머신러닝의 결과도 좋을 수가 없다

파이썬과 R 기반의 머신러닝 비교

파이썬, R > C, C++, JAVA

↳ 개발 생산성, 지원 패키지, 생태계

⇒ 주로 머신러닝에서는 파이썬과 R을 이용

< 파이썬 >

< R >

- 다양한 영역에서 사용되는 개발 전문 프로그램 언어
- 직관적인 문법, 객체지향과 함수형 프로그래밍을 모두 포괄, 다양한 라이브러리
- 통계 분석을 위해 특화된 언어, 다양하고 많은 통계 패키지 보유
-



2. 파이썬 머신러닝 생태계를 구성하는 주요 패키지

파이썬 기반의 머신러닝을 익히기 위해 필요한 패키지는 다음과 같음

▼ 머신러닝 패키지

사이킷런(Scikit - Learn)가 가장 대표적 - 데이터 마이닝 기반의 머신러닝에서 독보적
텐서플로, 케라스 등 전문 딥러닝 라이브러리도 존재

▼ 행렬/선형대수/통계 패키지

넘파이(Numpy) - 행렬과 선형대수를 다루는 패키지, 사이킷런을 비롯한 많은 머신러닝
패키지가 넘파이 기반으로 되어 있음

사이파이(Scipy) - 자연과학과 통계를 위한 다양한 패키지

▼ 데이터 핸들링

판다스 - 2차원 데이터 처리에 특화, 넘파이보다 훨씬 편리하게 데이터 처리를 위한 많은
기능 제공

▼ 시각화

맷플롯립 - 익히기 번거롭고, 디자인 측면에서 투박한 면이 있음, 이를 보완하기 위한 여
러 시각화 패키지가 출시되고 있음(대표적으로 시본이 있음)

시본 - 맷플롯립을 기본으로 만들어졌으나 판다스와의 더 쉬운 연동, 더 함축적인 API,
다양한 그래프 및 차트 제공을 함

▼ 이 외의 서드파티 라이브러리

파이썬의 큰 장점은 많은 라이브러리가 오픈 소스로 존재한다는 것

▼ 주피터 노트북

아이파이썬 툴인 주피터 노트북 - 아이파이썬은 대화형 파이썬 툴을 지칭함

파이썬 머신러닝을 위한 S/W 설치

1. 아나콘다 설치(파이썬 기반의 머신러닝에 필요한 패키지들을 일괄적으로 설치가능 - 넘파이, 판다스, 맷플롯립, 시분, 주피터 노트북까지 함께 설치됨)
2. Microsoft Visual Studio Build Tools 설치

넘파이와 판다스에 대해 알아보자

머신러닝 애플리케이션을 파이썬으로 개발한다면, 대부분의 코드는 사이킷런의 머신러닝 알고리즘에 입력하기 위한 데이터의 추출/가공/변형, 원하는 차원 배열로의 변환을 포함해 머신러닝 알고리즘 처리 결과에 대한 다양한 가공 등으로 구성될 것

→ 이 같은 데이터 처리 부분은 넘파이와 판다스로 수행됨

+사이킷런이 넘파이기반에서 작성됐기 때문에 넘파이의 기본 툴을 이해하지 못하면 사이킷런을 제대로 활용하기 힘들

→ 따라서 넘파이와 판다스에 대한 이해가 부족하면 다른 머신러닝 코드에 대한 전체 맥락을 이해할 수 없는 경우가 많음



3. 넘파이

- 머신러닝의 주요 알고리즘은 선형대수와 통계 등에 기반함. Numerical Python을 의미하는 넘파이는 파이썬에서 선형대수 기반의 프로그램을 쉽게 만들 수 있도록 지원하는 대표적인 패키지
- 루프를 사용하지 않고 대량 데이터의 배열 연산을 가능하게 하므로 빠른 배열 연산 속도를 보장함.
- C/C++과 같은 저수준 언어 기반의 호환 API를 제공함. → 넘파이는 매우 빠른 배열 연산을 보장해 주지만, 파이썬 언어 자체가 가지는 수행 성능의 제약이 있으므로 수행 성능

이 중요한 부분은 C/C++ 기반의 코드로 작성하고, 이를 넘파이에서 호출하는 방식으로 쉽게 통합 가능(대표적으로 텐서플로가 있음)

- 다양한 데이터 핸들링 기능을 제공.(하지만 판다스의 편리성에는 미치지 못함)

넘파이 ndarray 개요

```
import numpy as np
```

- as np를 추가해 약어로 모듈을 표현
- 넘파이 기반 데이터 타입은 ndarray → ndarray를 이용해 넘파이에서 다차원 배열을 쉽게 생성하고 다양한 연산을 수행 가능

```
array1 = np.array([1,2,3])
print('array1 type:', type(array1))
print('array1 array 형태 | ', array1.shape)
```

[OUTPUT]

```
array1 type: <class 'numpy.ndarray'>
array1 array 형태: (3, )
```

- np.array - ndarray로 변환을 원하는 객체를 인자로 입력하면 ndarray를 반환함.
- ndarray.shape - ndarray의 차원과 크기를 튜플의 형태로 나타내줌, 1,2차원을 다르게 표현해줌

```
print('array1: {0}차원'.format(array1.ndim))
```

[OUTPUT]

```
array1: 1차원
```

- 각 array의 차원을 ndarray.ndim을 이용해 확인 가능

ndarray의 데이터 타입

- ndarray의 데이터값은 숫자, 문자열, 불 값 모두 가능 - 숫자형인 경우 int형, unsigned int형, float형, complex타입
- ndarray 내의 데이터 타입은 그 연산의 특성상 같은 데이터 타입만 가능 - 데이터 타입은 dtype 속성으로 확인 가능
- 서로 다른 데이터 타입을 가질 수 있는 리스트와는 다르게 ndarray 내의 데이터 타입은 같은 것만 가능하다고 했는데, 만약 다른 데이터 유형이 섞여있는 리스트를 ndarray로 변경하면 데이터 크기가 더 큰 데이터 타입으로 형 변환을 일괄 적용함
- ndarray 내 데이터값의 타입 변경도 astype() 메서드를 이용해 할 수 있음. - 메모리를 절약해야할 때 주로 이용

ndarray를 편리하게 생성하기 - arange, zeros, ones

1. arange()

- 0부터 함수 인자 값 -1까지의 값을 순차적으로 ndarray의 데이터값으로 변환해 줌.

2. zeros()

- 함수 인자로 튜플 형태의 shape 값을 입력하면 모든 값을 0으로 채운 해당 shape를 가진 ndarray를 반환

3. ones()

- 함수 인자로 튜플 형태의 shape 값을 입력하면 모든 값을 1로 채운 해당 shape를 가진 ndarray를 반환

ndarray의 차원과 크기를 변경하는 reshape()

reshape() 메서드는 ndarray를 특정 차원 및 크기로 변환함. - 변환을 원하는 크기를 함수 인자로 부여하기.

- 지정된 사이즈로 변경이 불가능하면 오류를 발생함 - (10,) 데이터를 (4,3) shape 형태로 변경 불가능

- -1을 인자로 사용하면 ndarray와 호환되는 새로운 shape로 변환해 줌. → reshape(-1,1)과 같은 형태로 자주 사용됨, reshape(-1,1)은 원본 ndarray가 어떤 형태라도 2차원이고, 여러 개의 로우를 가지되 반드시 1개의 칼럼을 가진 ndarray로 변환됨을 보장.

넘파이의 ndarray의 데이터 세트 선택하기 - 인덱싱(Indexing)

1. 특정한 데이터만 추출 - 원하는 위치의 인덱스 값을 지정하면 해당 위치의 데이터가 반환됨
2. 슬라이싱 - 연속된 인덱스상의 ndarray를 추출하는 방식
3. 팬시 인덱싱 - 일정한 인덱싱 집합을 리스트 또는 ndarray 형태로 지정해 해당 위치에 있는 데이터의 ndarray를 반환
4. 불린 인덱싱 - 특정 조건에 해당하는지의 여부인 T/F 값 인덱싱 집합을 기반으로 True에 해당하는 인덱스 위치에 있는 데이터의 ndarray를 반환

단일 값 추출

1개의 데이터값을 선택하려면 ndarray 객체에 해당하는 위치의 인덱스 값을 [] 안에 입력

- 단일 인덱스를 이용해 ndarray 내의 데이터값도 간단히 수정 가능
- 1차원과 2차원 ndarray에서의 데이터 접근의 차이는 2차원의 경우 콤마로 분리된 인덱스를 통해 접근
- ndarray의 경우 축(axis)를 사용함 → axis 0은 로우 방향의 축을 의미, axis 1은 칼럼 방향의 축을 의미

슬라이싱

':' 기호를 이용해 연속한 데이터를 슬라이싱해서 추출할 수 있음 - 슬라이싱 기호인 ':' 사이의 시작, 종료 인덱스는 생략이 가능함(생략하면 맨 처음 또는 마지막 인덱스로 인식됨)

- 2차원 ndarray 슬라이싱은 콤마로 인덱스를 지칭하는 부분만 1차원과 다름 - 로우나 칼럼 축 한쪽에만 슬라이싱을 적용하고, 다른 쪽 축에는 단일 값 인덱스를 적용해도 됨

팬시 인덱싱

팬시 인덱싱은 리스트나 ndarray로 인덱스 집합을 지정하면 해당 위치의 인덱스에 해당하는 ndarray를 반환하는 인덱싱 방식

불린 인덱싱

불린 인덱싱은 조건 필터링과 검색을 동시에 할 수 있기 때문에 자주 사용됨. → ndarray의 인덱스를 지정하는 [] 내에 조건문을 그대로 기재하기만 하면 됨

▼ 불린 인덱싱이 동작하는 단계

Step 1: array1d > 5와 같이 ndarray의 필터링 조건을 []안에 기재

Step 2: False 값은 무시하고 True 값에 해당하는 인덱스값만 저장(유의해야 할 사항은 True 값 자체인 1을 저장하는 것이 아니라 True값을 가진 인덱스를 저장한다는 것)

Step 3: 저장된 인덱스 데이터 세트로 ndarray 조회

행렬의 정렬 - sort()와 argsort()

행렬 정렬

np.sort()	ndarray.sort()
• 넘파이에서 sort()를 호출 • 원 행렬은 그대로 유지한 채 원 행렬의 정렬된 행렬을 반환	• 행렬 자체에서 sort()를 호출 • 원 행렬 자체를 정렬한 형태로 변환하며 반환 값은 None

- 기본적으로 오름차순으로 행렬 내 원소를 정렬
- 내림차순으로 정렬하기 위해서는[::-1]을 적용
- 행렬이 2차원 이상인 경우에는 axis 축 값 설정을 통해 로우 방향 또는 칼럼 방향으로 정렬을 수행 가능

정렬된 행렬의 인덱스를 반환하기

- 원본 행렬이 정렬되었을 때 기존 원본 행렬의 원소에 대한 인덱스를 필요로 할 때 np.argsort()를 이용
- np.argsort()는 정렬 행렬의 원본 행렬 인덱스를 ndarray 형으로 반환

선형대수 연산 - 행렬 내적과 전치 행렬 구하기

행렬 내적(행렬 곱)

- 두 행렬 A와 B의 내적은 np.dot()을 이용해 계산이 가능
- 행렬 내적의 특성으로 왼쪽 행렬의 열 개수와 오른쪽 행렬의 행 개수가 동일해야 내적 연산이 가능

전치 행렬

- 원 행렬에서 행과 열 위치를 교환한 원소로 구성된 행렬을 그 행렬의 전치 행렬이라 함
- 넘파이의 transpose()를 이용해 전치 행렬을 구할 수 있음



4. 데이터 핸들링 - 판다스

판다스의 핵심 객체인 DataFrame은 여러 개의 행과 열로 이뤄진 2차원 데이터를 담는 구조체이다.

- Index - RDBMS의 PK처럼 개별 데이터를 고유하게 식별하는 Key값
- Series - DataFrame과의 가장 큰 차이는 Series는 칼럼이 하나뿐인 데이터 구조체이고, DataFrame은 칼럼이 여러 개인 데이터 구조체임

→ 즉, DataFrame은 여러 개의 Series로 이뤄졌다고 볼 수 있음

판다스 시작 - 파일을 DataFrame으로 로딩, 기본 API

```
import pandas as pd
```

캐글에서 데이터 파일을 다운 받아서 활용

- `DataFrame.head()`는 `DataFrame`의 맨 앞에 있는 N개의 로우를 반환함
- `DataFrame` 객체의 `shape` 변수를 이용하면 행과 열의 크기를 알 수 있음 - `shape`는 `DataFrame`의 행과 열을 튜플 형태로 반환함

1. `info()` - 총 데이터 건수와 데이터 타입, Null 건수를 알 수 있음
2. `describe()` - 칼럼별 숫자형 데이터값의 n-percentile 분포도, 평균값, 최댓값, 최솟값을 나타냄, 정확한 분포도를 알 수는 없지만 개략적인 수준의 분포도를 확인 할 수 있어 유용

DataFrame과 리스트, 딕셔너리, 넘파이 ndarray 상호 변환

넘파이 ndarray, 리스트, 딕셔너리를 DataFrame으로 변환하기

- `DataFrame`은 리스트와 넘파이 ndarray와 달리 칼럼명을 가지고 있음. → 리스트와 넘파이 ndarray보다 편하게 데이터 핸들링 가능
- 칼럼명 지정(지정하지 않으면 자동 지정) → `DataFrame` 객체의 생성 인자 `data`는 리스트나 딕셔너리 또는 넘파이 ndarray를 입력받음 → 생성 인자 `columns`는 칼럼명 리스트를 입력받아서 `DataFrame` 생성
- 2차원 이하의 데이터들만 `DataFrame`으로 변환 가능
- 딕셔너리를 변환할 때 딕셔너리의 키는 칼럼명으로, 딕셔너리의 값은 키에 해당하는 칼럼 데이터로 변환됨

DataFrame을 넘파이 ndarray, 리스트, 딕셔너리로 변환하기

- `DataFrame`을 넘파이 ndarray로 변환하는 것은 `DataFrame` 객체의 `values`를 이용해 쉽게 가능
- 리스트로의 변환은 `values`로 얻은 ndarray에 `tolist()`를 호출하면 됨
- 딕셔너리로의 변환은 `DataFrame` 객체의 `to_dict*` 메서드를 호출하는데, 인자로 'list'를 입력하면 딕셔너리의 값이 리스트형으로 반환됨

DataFrame의 칼럼 데이터 세트 생성과 수정

- 생성과 수정 또한 [] 연산자를 이용해 쉽게 할 수 있음 → DataFrame[] 내에 새로운 컬럼명을 입력하고 값을 할당해주기
- 기존 컬럼 값도 일괄적으로 업데이트 가능 → DataFrame[] 내에 컬럼명으로 입력한 뒤에 값을 할당해주기

DataFrame 데이터 삭제

drop()을 이용

- axis: DataFrame의 로우를 삭제할 때는 axis=0, 컬럼을 삭제할 때는 axis=1로 설정
- 원본 DataFrame은 유지하고 드롭된 DataFrame을 새롭게 개체 변수로 받고 싶다면 inplace=False로 설정
- 원본 DataFrame에 드롭된 결과를 적용할 경우에는 inplace=True를 적용
- 원본 DataFrame에서 드롭된 DataFrame을 다시 원본 DataFrame 객체 변수로 할당하면 원본 DataFrame에서 드롭된 결과를 적용할 경우와 같음

Index 개체

판다스의 Index개체는 RDBMS의 PK와 유사하게 DataFrame, Series의 레코드를 고유하게 식별하는 객체

- Series 객체는 Index 객체를 포함하지만 Series 객체에 연산 함수를 적용할 때 Index는 연산에서 제외
- DataFrame 및 Series에 reset_index()를 수행하면 새롭게 인덱스를 연속 숫자 형으로 할당함

데이터 셀렉션 및 필터링

DataFrame의 [] 연산자

- DataFrame 뒤의 [] 연산자는 넘파이의 []나 Series의 []와 다름
- DataFrame 뒤의 [] 내 입력값은 컬럼명을 지정해 컬럼 지정 연산에 사용하거나 불린 인덱스 용도로만 사용
- DataFrame[0:2]와 같은 슬라이싱 연산으로 데이터 추출하는 방법은 권장하지 않음

DataFrame.iloc[] 연산자

로우나 칼럼을 지정하여 데이터를 선택할 수 있는 인덱싱 방식으로 iloc[]과 loc[]가 있다.

- iloc[]은 위치기반 인덱싱 방식으로, loc[]은 명칭 기반 인덱싱 방식으로 동작함
- iloc[행 위치 정숫값, 열 위치 정숫값]과 같이 DataFrame의 행 위치와 열 위치를 좌표 정숫값 형태로 입력하여 해당 위치를 데이터를 가져올 수 있음 - 인덱스 값이나 칼럼 명을 입력하면 오류
- iloc[]는 열 위치에 -1을 입력하여 DataFrame의 가장 마지막 열 데이터를 가져오는 데 자주 사용

DataFrame.loc[] 연산자

- loc[인덱스값, 칼럼명]과 같은 형식으로 데이터 추출 가능
- loc[]에 슬라이싱 기호를 적용하면 종료 값 -1이 아니라 종료 값까지 포함하는 것을 의미

불린 인덱싱

- [], loc[] 에서 지원, iloc[]에서 미지원
- and 조건일 때는 &, or조건일 때 |, Not 조건일 때 ~

정렬, Aggregation 함수, GroupBy 적용

DataFrame, Series의 정렬 - sort_values()

- by로 특정 칼럼을 입력하면 해당 칼럼으로 정렬을 수행
- ascending = True로 설정하면 오름차순
- inplace = False 로 설정하면 sort_values()를 호출한 DataFrame은 그대로 유지하며 정렬된 DataFrame을 결과로 반환

Aggregation 함수 적용

min(), max(), sum(), count()와 같은 aggregation 함수의 적용은 RDBMS SQL의 aggregation 함수와 유사, 다만 DataFrame의 경우 DataFrame에서 바로 aggregation

을 호출할 경우 모든 칼럼에 해당 aggregation을 적용

groupby() 적용

DataFrame의 groupby() 사용 시 입력 파라미터 by에 칼럼을 입력하면 대상 칼럼으로 groupby됨. DataFrame에 groupby()를 호출하면 DataFrameGroupBy라는 또 다른 형태의 DataFrame을 반환

- 차이점1: DataFrame에 groupby()를 호출해 반환된 결과에 aggregation 함수를 호출하면 groupby() 대상 칼럼을 제외한 모든 칼럼에 해당 aggregation 함수를 적용
- 차이점2: 여러 개의 aggregation 함수명을 DataFrameGroupBy 객체의 agg()내에 인자로 입력해서 사용

⇒ SQL의 group by보다 유연성이 떨어짐...

결손 데이터 처리하기

결손 데이터는 칼럼에 값이 없는, 즉 NULL인 경우를 의미, 이를 넘파이의 NaN으로 표시

- 머신러닝 알고리즘은 이 NaN 값을 처리하지 않음 → 다른 값으로 대체 필요
- NaN 값은 함수 연산 시 제외(평균, 총합 등)
- NaN 여부를 확인하는 API는 isna(), NaN 값을 다른 값으로 대체하는 API는 fillna()

isna()로 결손 데이터 여부 확인

isna()를 수행하면 모든 칼럼의 값이 NaN인지 아닌지를 True나 False로 알려줌

→ 결손 데이터의 개수는 isna() 결과에 sum() 함수를 추가해 구할 수 있음

fillna()로 결손 데이터 대체하기

fillna()를 이용하면 결손 데이터를 다른 값으로 대체 가능

apply lambda 식으로 데이터 가공

apply 함수에 lambda 식을 결합해서 레코드별로 데이터를 가공하는 기능을 제공

- lambda는 파이썬에서 함수형 프로그래밍을 지원하기 위해 만들어짐 → 함수의 선언과 함수 내의 처리를 한 줄의 식으로 쉽게 변환하는 식



5. 정리

실제로 ML 모델을 생성하고 예측을 수행하는 데 있어서 ML 알고리즘이 차지하는 비중보다 데이터를 전처리하고 적절한 피처를 가공/추출하는 부분이 훨씬 많은 비중을 차지함

⇒ 머신러닝 생태계를 이해하는 것 또한 매우 중요하다!!